

Sitting across from an Executive Director (ED) on the **Cash Equities Central Risk Book (CRB)** desk at Nomura, your understanding of econometric diagnostics cannot be superficial. In the CRB, you manage residual risks from large client internalizations and block trades. If your asset pricing or risk factor models suffer from unaddressed heteroskedasticity, your standard errors are structurally distorted. This invalidates your t -statistics, compromises tracking error projections, and misallocates scarce risk capital.

Below is the definitive, first-principles mathematical and programmatic breakdown of heteroskedasticity diagnostics and robust structural fixes required for this front-office quantitative research role.

1. Mathematical Foundations & Breakdown from First Principles

The Classical Linear Regression Model (CLRM) Setup

Consider a cross-sectional equity factor model mapping asset returns to risk factors across our universe:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

Where:

- $\mathbf{y} \in \mathbb{R}^{n \times 1}$ is the vector of asset excess returns for n equities.
- $\mathbf{X} \in \mathbb{R}^{n \times K}$ is the design matrix of factor exposures (e.g., style, sector, and risk exposures), containing a column of ones for the intercept.
- $\boldsymbol{\beta} \in \mathbb{R}^{K \times 1}$ is the vector of true underlying factor risk premiums.
- $\boldsymbol{\epsilon} \in \mathbb{R}^{n \times 1}$ is the vector of unobservable structural perturbations (idiosyncratic risk).

Painstaking Derivation of the Ordinary Least Squares (OLS) Estimator

To find the empirical factor premium vector $\hat{\boldsymbol{\beta}}$, we minimize the objective function representing the residual sum of squares (S):

$$S(\boldsymbol{\beta}) = \boldsymbol{\epsilon}^\top \boldsymbol{\epsilon} = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

Expanding the matrix multiplication step-by-step:

$$S(\boldsymbol{\beta}) = \mathbf{y}^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\boldsymbol{\beta} - (\mathbf{X}\boldsymbol{\beta})^\top \mathbf{y} + (\mathbf{X}\boldsymbol{\beta})^\top \mathbf{X}\boldsymbol{\beta}$$

Since $\mathbf{y}^\top \mathbf{X}\boldsymbol{\beta}$ is a scalar (1×1), it is equivalent to its own transpose:

$$(\mathbf{y}^\top \mathbf{X}\boldsymbol{\beta})^\top = \boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{y}$$

Substituting this equality yields:

$$S(\boldsymbol{\beta}) = \mathbf{y}^\top \mathbf{y} - 2\boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\beta}$$

To find the minimum, compute the first-order partial derivative with respect to the vector $\boldsymbol{\beta}$:

$$\frac{\partial S}{\partial \boldsymbol{\beta}} = \mathbf{0} - 2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\beta}$$

Setting this gradient vector equal to $\mathbf{0}$ defines the normal equations:

$$2\mathbf{X}^\top \mathbf{X}\hat{\boldsymbol{\beta}} = 2\mathbf{X}^\top \mathbf{y}$$

$$\mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} = \mathbf{X}^\top \mathbf{y}$$

Assuming the cross-product matrix $\mathbf{X}^\top \mathbf{X}$ is non-singular (invertible, meaning no perfect multi-collinearity in our factor exposures), left-multiply by $(\mathbf{X}^\top \mathbf{X})^{-1}$:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Derivation of the Sampling Variance-Covariance Matrix

To analyze standard errors, substitute the true structural process $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$ back into our derived estimator:

$$\begin{aligned}\hat{\boldsymbol{\beta}} &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}) \\ \hat{\boldsymbol{\beta}} &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X}\boldsymbol{\beta} + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon} \\ \hat{\boldsymbol{\beta}} &= \boldsymbol{\beta} + (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon}\end{aligned}$$

The conditional variance-covariance matrix of our estimator is defined as:

$$\text{Var}(\hat{\boldsymbol{\beta}}|\mathbf{X}) = \mathbb{E} \left[(\hat{\boldsymbol{\beta}} - \boldsymbol{\beta})(\hat{\boldsymbol{\beta}} - \boldsymbol{\beta})^\top | \mathbf{X} \right]$$

Substitute the expression for $(\hat{\boldsymbol{\beta}} - \boldsymbol{\beta})$:

$$\text{Var}(\hat{\boldsymbol{\beta}}|\mathbf{X}) = \mathbb{E} \left[((\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon}) ((\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon})^\top | \mathbf{X} \right]$$

Using the matrix transposition property $(\mathbf{ABC})^\top = \mathbf{C}^\top \mathbf{B}^\top \mathbf{A}^\top$:

$$\text{Var}(\hat{\boldsymbol{\beta}}|\mathbf{X}) = \mathbb{E} \left[(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{\epsilon} \boldsymbol{\epsilon}^\top \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} | \mathbf{X} \right]$$

Since $\mathbf{X}$ is deterministic conditional on itself, pull the design matrices outside the expectation operator:

$$\text{Var}(\hat{\boldsymbol{\beta}}|\mathbf{X}) = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbb{E}[\boldsymbol{\epsilon} \boldsymbol{\epsilon}^\top | \mathbf{X}] \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1}$$

Let $\boldsymbol{\Omega} = \mathbb{E}[\boldsymbol{\epsilon} \boldsymbol{\epsilon}^\top | \mathbf{X}]$ represent the true covariance matrix of our errors.

Case A: The Homoskedastic Assumption

If the errors are identically distributed and mutually independent, then $\boldsymbol{\Omega} = \sigma^2 \mathbf{I}_n$:

$$\text{Var}(\hat{\boldsymbol{\beta}}|\mathbf{X}) = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top (\sigma^2 \mathbf{I}_n) \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1}$$

Factor out the scalar variance component σ^2 :

$$\text{Var}(\hat{\boldsymbol{\beta}}|\mathbf{X}) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{X} (\mathbf{X}^\top \mathbf{X})^{-1} = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$$

Case B: The Heteroskedastic Reality

If error variances differ across assets, then $\boldsymbol{\Omega}$ is a diagonal matrix of non-equal variances:

$$\mathbf{\Omega} = \begin{bmatrix} \sigma_1^2 & 0 & \dots & 0 \\ 0 & \sigma_2^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_n^2 \end{bmatrix}$$

Under this condition, the expression cannot simplify, leaving us with the **"Sandwich" Covariance Estimator**:

$$\text{Var}(\hat{\beta}|\mathbf{X}) = (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{\Omega} \mathbf{X}) (\mathbf{X}^\top \mathbf{X})^{-1}$$

Structural Failure on the CRB Desk

If you use the classical formula $\hat{\sigma}^2(\mathbf{X}^\top \mathbf{X})^{-1}$ when the system is governed by $\mathbf{\Omega}$, your standard errors are structurally biased.

In equity markets, smaller, highly volatile stocks exhibit wider residual distributions than multi-billion-dollar large caps. Treating them equally causes standard software packages to underestimate the standard errors of your factor loadings. This inflates your t -statistics ($t = \frac{\hat{\beta}_k}{\text{SE}}$), leading you to mistake random noise for valid alpha signals.

2. Comprehensive Mathematical Derivation of Diagnostic Tests

The Breusch-Pagan Test

The Breusch-Pagan test checks whether the variance of your residuals depends linearly on a set of independent variables:

$$\sigma_i^2 = \gamma_0 + \gamma_1 z_{i1} + \gamma_2 z_{i2} + \dots + \gamma_m z_{im}$$

To test for homoskedasticity, we evaluate the joint null hypothesis:

$$H_0 : \gamma_1 = \gamma_2 = \dots = \gamma_m = 0$$

Step-by-Step Derivation of the Test Statistic

1. Run the primary factor model via OLS and isolate the sample empirical residuals:

$$\hat{\epsilon}_i = y_i - \mathbf{x}_i^\top \hat{\beta}$$

2. Construct the auxiliary regression. Since the true variance σ_i^2 is unobservable, we use the squared empirical residuals $\hat{\epsilon}_i^2$ as an unbiased operational proxy:

$$\hat{\epsilon}_i^2 = \gamma_0 + \gamma_1 x_{i1} + \gamma_2 x_{i2} + \dots + \gamma_K x_{iK} + v_i$$

3. Extract the coefficient of determination (R_{aux}^2) from this auxiliary regression:

$$R_{\text{aux}}^2 = \frac{\sum_{i=1}^n (\hat{\epsilon}_i^2 - \bar{\hat{\epsilon}^2})^2}{\sum_{i=1}^n (\hat{\epsilon}_i^2 - \bar{\hat{\epsilon}^2})^2}$$

Where $\hat{\epsilon}_i^2$ represents the fitted value from the auxiliary equation. 4. The Breusch-Pagan test statistic maps directly to a Lagrange Multiplier (LM) framework:

$$LM = n \cdot R_{\text{aux}}^2$$

Asymptotically, under the null hypothesis of constant variance, this scaled metric converges to a Chi-squared distribution with degrees of freedom equal to the number of active regressors in the auxiliary step (K):

$$n \cdot R_{\text{aux}}^2 \xrightarrow{d} \chi_K^2$$

The White Test

The Breusch-Pagan test can fail to flag non-linear heteroskedasticity. The White test resolves this by mapping non-constant variances directly to the original regressors, their unique squares, and cross-product interaction terms.

The Auxiliary Equation Architecture

For a model tracking two primary risk factors (x_1 and x_2), the auxiliary regression is structured as:

$$\hat{\epsilon}_i^2 = \gamma_0 + \gamma_1 x_{i1} + \gamma_2 x_{i2} + \gamma_3 x_{i1}^2 + \gamma_4 x_{i2}^2 + \gamma_5 (x_{i1} x_{i2}) + v_i$$

For a general model with K variables, the total number of parameters to estimate in the auxiliary regression (P) equals:

$$P = \frac{K(K + 3)}{2}$$

The test statistic is calculated as:

$$W = n \cdot R_{\text{aux}}^2 \xrightarrow{d} \chi_P^2$$

Trade-offs for the CRB Risk Desk

- **Advantage:** The White test is completely agnostic to the underlying functional form of the heteroskedasticity. It captures both linear relationships and complex interactions.
- **Disadvantage:** It suffers from the curse of dimensionality. If our risk book uses a Barra framework with 60 distinct risk factor exposures, $P = \frac{60 \times 63}{2} = 1,890$ parameters. This devalues your degrees of freedom and erodes the statistical power of the test unless your cross-sectional sample size (n) is exceptionally large.

3. Institutional Context & Architectural Fixes for the CRB Desk

Heteroskedasticity is a natural structural feature of multi-factor asset pricing models, driven by three primary market dynamics:

Structural Cause	Financial Phenomenon	Operational Impact on Risk Models
Market Capitalization Scaling	Small-cap asset returns exhibit significantly higher idiosyncratic volatility than mega-cap stocks.	A uniform OLS framework allows volatile micro-cap residuals to distort global risk pricing.

Structural Cause	Financial Phenomenon	Operational Impact on Risk Models
Sector Volatility Stratification	Tech, biotech, and crypto-correlated sectors display wider variance bands than utilities or consumer staples.	Unadjusted factor regressions generate mispriced risk signals across sector buckets.
Regime-Dependent Clustering	Asset return volatility clusters during market shocks or liquidity crunches.	Assuming constant error variance causes models to understate tail-risk metrics.

Architectural Fixes

Fix 1: Weighted Least Squares (WLS) / Generalized Least Squares (GLS)

If the variance profiles follow a known structure—for example, if idiosyncratic variance scales inversely with market capitalization ($\sigma_i^2 = \frac{\sigma^2}{w_i}$, where w_i is proportional to market cap)—we transform the system using a weight matrix $\mathbf{W} = \mathbf{\Omega}^{-1/2}$.

Left-multiply the entire system by $\mathbf{W}$:

$$\mathbf{W}\mathbf{y} = \mathbf{W}\mathbf{X}\boldsymbol{\beta} + \mathbf{W}\boldsymbol{\epsilon}$$

The variance of the transformed error term equals:

$$\text{Var}(\mathbf{W}\boldsymbol{\epsilon}|\mathbf{X}) = \mathbf{W}\mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top|\mathbf{X}]\mathbf{W}^\top = \mathbf{\Omega}^{-1/2}\mathbf{\Omega}\mathbf{\Omega}^{-1/2} = \mathbf{I}_n$$

This standardizes the error variance, restoring the efficiency of the OLS estimator. The resulting GLS framework is expressed as:

$$\hat{\boldsymbol{\beta}}_{\text{GLS}} = (\mathbf{X}^\top \mathbf{\Omega}^{-1} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{\Omega}^{-1} \mathbf{y}$$

Fix 2: Heteroskedasticity-Consistent (HC) Robust Standard Errors

If the exact functional form of $\mathbf{\Omega}$ is unknown, we preserve our OLS point estimates $\hat{\boldsymbol{\beta}}$ but replace the standard error calculation with a Heteroskedasticity-Consistent (HC) covariance matrix:

$$\text{Est.Var}(\hat{\boldsymbol{\beta}}) = (\mathbf{X}^\top \mathbf{X})^{-1} \left(\mathbf{X}^\top \hat{\mathbf{\Omega}} \mathbf{X} \right) (\mathbf{X}^\top \mathbf{X})^{-1}$$

We select the operational structure of the diagonal matrix $\hat{\mathbf{\Omega}} = \text{diag}(\hat{\sigma}_1^2, \dots, \hat{\sigma}_n^2)$ based on sample size and leverage constraints:

- **HC0:** Uses the raw squared residuals: $\hat{\sigma}_i^2 = \hat{\epsilon}_i^2$. This estimator is simple but biased downward in small samples.
- **HC1:** Applies a degrees-of-freedom correction factor: $\hat{\sigma}_i^2 = \frac{n}{n-K} \hat{\epsilon}_i^2$.
- **HC2:** Adjusts for the structural leverage (h_i) of individual observations: $\hat{\sigma}_i^2 = \frac{\hat{\epsilon}_i^2}{1-h_i}$, where h_i represents the diagonal entries of the projection "hat" matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$.
- **HC3:** Closes small-sample estimation gaps by dividing by the squared leverage term:

$$\hat{\sigma}_i^2 = \frac{\hat{\epsilon}_i^2}{(1 - h_i)^2}$$

Why HC3 is preferred on the CRB desk: Observations with high leverage (h_i) can heavily influence OLS point estimates. By scaling the squared residual by $(1 - h_i)^2$, HC3 penalizes high-leverage positions more aggressively. This protects risk models from understating standard errors when concentrated institutional trades impact inventory liquidity.

4. Production-Grade Python Engine & Interactive Visualization Panel

This modular script implements OLS estimation, calculates the Breusch-Pagan and White diagnostic tests from first principles, constructs the robust standard error frameworks (HC0–HC3), and renders an interactive diagnostic dashboard using Plotly.

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import plotly.graph_objects as go
from plotly.subplots import make_subplots

class CRBRegressionEngine:
    """
    Production-grade econometric engine designed for cash equities cross-sectional
    factor models. Implements OLS, analytical diagnostics, and robust corrections.
    """
    def __init__(self, y: np.ndarray, X: np.ndarray):
        self.y = y.flatten()
        # Add intercept column to the design matrix if not already present
        if not np.all(X[:, 0] == 1):
            self.X = np.hstack([np.ones((X.shape[0], 1)), X])
        else:
            self.X = X

        self.n, self.K = self.X.shape
        self._run_ols()

    def _run_ols(self):
        """Executes core matrix algebraic OLS operations."""
        xtx = self.X.T @ self.X
        self.xtx_inv = np.linalg.inv(xtx)
        self.beta = self.xtx_inv @ self.X.T @ self.y
        self.fitted = self.X @ self.beta
        self.residuals = self.y - self.fitted
        self.rss = np.sum(self.residuals ** 2)
        self.sigma_sq_omitted = self.rss / (self.n - self.K)

        # Diagonal elements of the projection matrix (Leverage values)
        self.h = np.diag(self.X @ self.xtx_inv @ self.X.T)

    def compute_standard_errors(self, method: str = 'classical') -> np.ndarray:
        """
```

```

    Calculates parameter standard errors across classical and robust
    frameworks.
    """
    if method.lower() == 'classical':
        vcov = self.sigma_sq_omitted * self.xtx_inv
        return np.sqrt(np.diag(vcov))

    # Extract squared residuals for the robust sandwich core
    e_sq = self.residuals ** 2

    if method.upper() == 'HC0':
        omega = np.diag(e_sq)
    elif method.upper() == 'HC1':
        omega = np.diag(e_sq * (self.n / (self.n - self.K)))
    elif method.upper() == 'HC2':
        omega = np.diag(e_sq / (1.0 - self.h))
    elif method.upper() == 'HC3':
        omega = np.diag(e_sq / ((1.0 - self.h) ** 2))
    else:
        raise ValueError(f"Unknown estimation framework: {method}")

    vcov = self.xtx_inv @ (self.X.T @ omega @ self.X) @ self.xtx_inv
    return np.sqrt(np.diag(vcov))

def run_breusch_pagan_test(self) -> dict:
    """
    Computes the Breusch-Pagan Lagrange Multiplier test from scratch.
    """
    # Step 1: Use squared OLS residuals as the dependent variable
    y_aux = self.residuals ** 2

    # Step 2: Use original regressors (excluding intercept) for auxiliary step
    X_aux = self.X

    # Run auxiliary OLS estimation
    xtx_aux = X_aux.T @ X_aux
    beta_aux = np.linalg.inv(xtx_aux) @ X_aux.T @ y_aux
    fitted_aux = X_aux @ beta_aux

    # Calculate auxiliary R-squared
    ss_total = np.sum((y_aux - np.mean(y_aux)) ** 2)
    ss_res = np.sum((y_aux - fitted_aux) ** 2)
    r_sq_aux = 1.0 - (ss_res / ss_total)

    # Calculate LM test statistic
    bp_stat = self.n * r_sq_aux
    # Degrees of freedom equals total regressors minus the auxiliary intercept
    df = self.K - 1
    p_val = stats.chi2.sf(bp_stat, df=df)

    return {"statistic": bp_stat, "df": df, "p_value": p_val}

def run_white_test(self) -> dict:
    """

```

```

Computes the White test from first principles by expanding the
auxiliary design matrix with square and cross-product interaction terms.
"""
y_aux = self.residuals ** 2

# Extract unique non-intercept columns
X_raw = self.X[:, 1:]
raw_k = X_raw.shape[1]

# Build interaction and squared terms
aux_list = [np.ones((self.n, 1)), X_raw]

# Add pure squared transformations
for i in range(raw_k):
    aux_list.append(X_raw[:, i:i+1] ** 2)

# Add unique cross-product combinations
for i in range(raw_k):
    for j in range(i + 1, raw_k):
        aux_list.append(X_raw[:, i:i+1] * X_raw[:, j:j+1])

X_white = np.hstack(aux_list)
total_p = X_white.shape[1]

# Run auxiliary regression
try:
    xtx_white = X_white.T @ X_white
    beta_white = np.linalg.inv(xtx_white) @ X_white.T @ y_aux
    fitted_white = X_white @ beta_white

    ss_total = np.sum((y_aux - np.mean(y_aux)) ** 2)
    ss_res = np.sum((y_aux - fitted_white) ** 2)
    r_sq_white = 1.0 - (ss_res / ss_total)

    w_stat = self.n * r_sq_white
    df = total_p - 1
    p_val = stats.chi2.sf(w_stat, df=df)
except np.linalg.LinAlgError:
    # Handle collinearity issues in structural dummy terms
    return {"statistic": np.nan, "df": 0, "p_value": 1.0}

return {"statistic": w_stat, "df": df, "p_value": p_val}

def generate_diagnostic_plots(self) -> go.Figure:
    """
    Generates an interactive dashboard to monitor heteroskedasticity and
    compare standard errors across corrections.
    """
    bp_res = self.run_breusch_pagan_test()
    w_res = self.run_white_test()

    methods = ['Classical', 'HC0', 'HC1', 'HC2', 'HC3']
    se_profiles = {m: self.compute_standard_errors(method=m) for m in methods}

```



```

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=(
        "Residual Scale-Location Plot",
        "Distribution of Squared Residuals vs Leverage",
        "Standard Error Inflation Metrics",
        "Parameter Point Estimates & Confidence Bands"
    ),
    vertical_spacing=0.18, horizontal_spacing=0.12
)

# Panel 1: Scale-Location Plot
fig.add_trace(
    go.Scatter(
        x=self.fitted, y=np.abs(self.residuals),
        mode='markers', name='Assets',
        marker=dict(color='#003366', size=6, opacity=0.7)
    ), row=1, col=1
)
fig.update_xaxes(title_text="Fitted Expected Returns", row=1, col=1)
fig.update_yaxes(title_text="|Residual Return|", row=1, col=1)

# Panel 2: Squared Residuals vs Leverage
fig.add_trace(
    go.Scatter(
        x=self.h, y=self.residuals**2,
        mode='markers', name='Leverage Points',
        marker=dict(color='#D96B27', size=6, opacity=0.7)
    ), row=1, col=2
)
fig.update_xaxes(title_text="Leverage Score (Hat Diagonal)", row=1, col=2)
fig.update_yaxes(title_text="Squared Residuals", row=1, col=2)

# Panel 3: Standard Error Inflation
for i in range(self.K):
    fig.add_trace(
        go.Bar(
            x=methods, y=[se_profiles[m][i] for m in methods],
            name=f"Beta_{i} SE",
            visible=True if i <= 2 else 'legendonly'
        ), row=2, col=1
    )
fig.update_xaxes(title_text="Estimation Framework", row=2, col=1)
fig.update_yaxes(title_text="Standard Error Value", row=2, col=1)

# Panel 4: HC3 Bound Shifts
beta_idx = np.arange(self.K)
se_classical = se_profiles['Classical']
se_hc3 = se_profiles['HC3']

fig.add_trace(
    go.Scatter(
        x=beta_idx, y=self.beta,
        mode='markers', name='Beta Point Estimate',

```

```

        marker=dict(color='black', size=8, symbol='diamond')
    ), row=2, col=2
)
fig.add_trace(
    go.Scatter(
        x=beta_idx, y=self.beta + 1.96 * se_classical,
        mode='markers', name='Classical 95% Upper Bound',
        marker=dict(color='red', size=6, symbol='triangle-up')
    ), row=2, col=2
)
fig.add_trace(
    go.Scatter(
        x=beta_idx, y=self.beta + 1.96 * se_hc3,
        mode='markers', name='HC3 Robust 95% Upper Bound',
        marker=dict(color='green', size=6, symbol='triangle-up')
    ), row=2, col=2
)
fig.update_xaxes(title_text="Factor Parameter Index", row=2, col=2)
fig.update_yaxes(title_text="Coefficient Value Spectrum", row=2, col=2)

meta_summary = (
    f"Breusch-Pagan p-value: {bp_res['p_value']:.5f} | "
    f"General White Test p-value: {w_res['p_value']:.5f}"
)

fig.update_layout(
    title=dict(
        text=f"<b>Nomura CRB Linear Diagnostics Matrix Panel</b><br><sup>
{meta_summary}</sup>",
        x=0.05, font=dict(size=16, family="Arial")
    ),
    template="plotly_white", height=780, width=1150, showlegend=True
)

return fig

if __name__ == "__main__":
    # Generate structured cross-sectional stock data with heteroskedastic errors
    np.random.seed(13458) # Set seed to the job code from NOMURA_JD.md
    n_assets = 300

    # Simulate factor exposures (e.g., Size, Value)
    factor_1 = np.random.normal(0, 1, n_assets)
    factor_2 = np.random.uniform(-2, 2, n_assets)
    X_matrix = np.column_stack([factor_1, factor_2])

    # Inject linear heteroskedasticity proportional to the second factor
    heteroskedastic_variance = 0.5 + 2.0 * (factor_2 + 2.0)**2
    idiosyncratic_noise = np.random.normal(0, np.sqrt(heteroskedastic_variance))

    # Calculate returns based on a true beta parameter vector of [1.5, 0.8, -1.2]
    true_beta = np.array([1.5, 0.8, -1.2])
    X_with_intercept = np.hstack([np.ones((n_assets, 1)), X_matrix])
    asset_returns = X_with_intercept @ true_beta + idiosyncratic_noise

```

```

# Run the quantitative diagnostic engine
engine = CRBRegressionEngine(y=asset_returns, X=X_matrix)
bp_output = engine.run_breusch_pagan_test()
white_output = engine.run_white_test()

print("="*65)
print("          NOMURA GLOBAL MARKETS COEFFICIENT DIAGNOSTICS          ")
print("="*65)
print(f"OLS Estimate Vector Summary: {engine.beta}")
print("-"*65)
print(f"Breusch-Pagan LM Score : {bp_output['statistic']:.4f} | p-value:
{bp_output['p_value']:.7f}")
print(f"White Test General Score: {white_output['statistic']:.4f} | p-value:
{white_output['p_value']:.7f}")
print("-"*65)

se_cls = engine.compute_standard_errors('classical')
se_hc3 = engine.compute_standard_errors('HC3')

for idx in range(len(se_cls)):
    print(f"Parameter [{idx}] Classical SE: {se_cls[idx]:.4f} | HC3 Robust SE:
{se_hc3[idx]:.4f} "
          f"-> Difference Inflation:
{((se_hc3[idx]/se_cls[idx])-1)*100:.1f}%")
    print("="*65)

# Display the dashboard layout
# dashboard_view = engine.generate_diagnostic_plots()
# dashboard_view.show()

```